

Abaqus jobs parallelization procedure in a SLURM cluster¹

José M. Pimentel; jpimentel@us.es

E.T.S.I., Universidad de Sevilla

September 16, 2025

¹With the PMTE-SC framework in mind

Preliminaries

- 1 The subprocess module must be imported into abaqus python.

Preliminaries

- 1 The subprocess module must be imported into abaqus python.
- 2 Whether the job parallelization technique is performed with Job Array Support² or by a master Python script, the number of concurrent jobs will be limited by the number of available Abaqus license tokens in the cluster.
- 3 Given condition (2), an alternative strategy is to stage job execution into separate batches, e.g. two or more job groups, depending on the available license tokens.
- 4 Each batch job must (preferably) have its own directory, to avoid interference with .lck and other files.

Filename	Filesize	Filetype
■ DCBuinte..._L237m114_N10_K15_scratch		File folder
■ DCBuinte..._L237m114_N11_K15_scratch		File folder
■ DCBuinte..._L237m114_N1_K15_scratch		File folder
■ DCBuinte..._L237m114_N2_K15_scratch		File folder
■ DCBuinte..._L237m114_N3_K15_scratch		File folder
■ DCBuinte..._L237m114_N4_K15_scratch		File folder
■ DCBuinte..._L237m114_N5_K15_scratch		File folder
■ DCBuinte..._L237m114_N6_K15_scratch		File folder
■ DCBuinte..._L237m114_N7_K15_scratch		File folder
■ DCBuinte..._L237m114_N8_K15_scratch		File folder

Application

```
Joblist=[]
processlist=[]
# ***ABAQUS JOB PARALLELIZATION APPROACH:***
abaqus_executable = '/opt/abaqus/Commands/abq2022'
for job in range(1,len(Uinterprops)-4):
    # Build a list of command arguments
    commandN = [
        abaqus_executable,
        'job=' + myJobN.name,
        'user=' + nameSUBR,
        'interactive']

    # subprocess.Popen launches the command in a new process
    # and the script immediately continues to the next line.
    process = subprocess.Popen(commandN, cwd=work_dir)
    processlist.append(process)
```

- * 'interactive' keyword is essential for a post-processing free of errors.
- * 'cwd' argument will guarantee each job is run in its working directory.

Wait for end of the jobs and post-process stages:

```
# --- LOOP 2: WAIT FOR JOBS TO FINISH ---  
    # This loop will pause the script until every job is done.  
    for i, proc in enumerate(processlist):  
        # .wait() method blocks execution until the process finishes  
        return_code = proc.wait()  
  
# --- LOOP 3: POST-PROCESSING ---  
    #  
    for job in Joblist:  
        # Open .odb files and extract simulation data.
```

Summary

PMTESC-3D computational efficiency pillars:

- Load increment bisection scheme: variable load-increment magnitude.
- Jobs parallelization, e.g. different model parameters for the same applied load.
- j_{ama} iterations: convergence of damage criteria within the user subroutine file; this approach is most compatible with the subprocess module command³:

```
process = subprocess.Popen(commandN, cwd=work_dir)
```

³As opposed to the `job.submit()` method of abaqus Python.

Acknowledgements

- The author expresses gratitude for useful comments by Alejandro Estefani (E.T.S.I., Sevilla), who suggested alternative solution strategies in parallelization allowed in a Linux cluster.